

Frontier Navigation Utilizing Turtle Bot 3 with A Star Based Planning and Non-Linear Positional Controller with Look Ahead Logic

Aim of the project:

1. Create a controller for a robot to follow a desired trajectory
2. Navigate an unknown environment to produce a map of the environment
3. Combine the controller and navigation to perform exploration of the environment

Setup:

1. The framework used was ROS2 Jazzy on WSL2
2. Simulator used was Gazebo Harmonic
3. Visualization was performed on Gazebo and Rviz2
4. The robot platform used was Turtle Bot 3
5. The world used was Turtle Bot 3 gazebo world, gazebo house and empty world
6. Empty world was used for pose-to-pose navigation and tracking a circular trajectory.
7. The video link is available on this google drive:
https://drive.google.com/drive/folders/1YqQXa9Y-e9lgGSmyM_u9Ron96vYLwuG?usp=sharing
8. To run the code, open a terminal for each of these commands and run them:
 - a. `ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py`
 - b. `ros2 launch slam_toolbox online_async_launch.py`
 - c. `rviz2`
 - d. `ros2 run controller frontier_explorer`
 - e. `ros2 run controller a_star_planner`
 - f. `ros2 run controller simple_controller`
9. Run the code in the above-mentioned order. If a_star planner is run before frontier explorer the frontier points don't get registered.

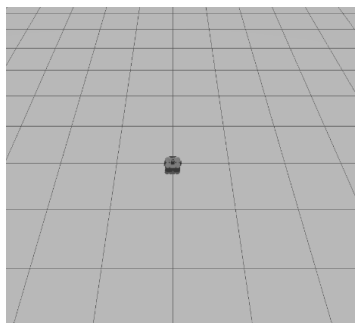


Figure: Gazebo empty world simulation environment with turtle bot 3

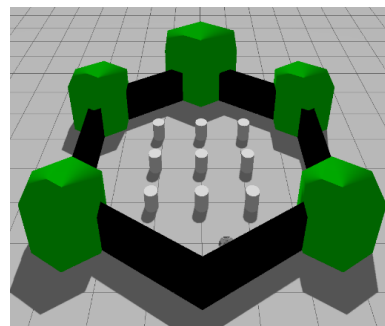


Figure: Gazebo world simulation environment with turtle bot 3

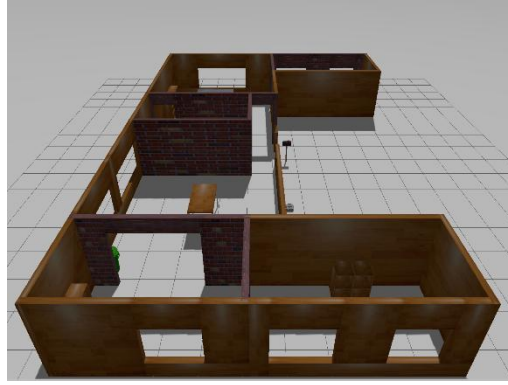
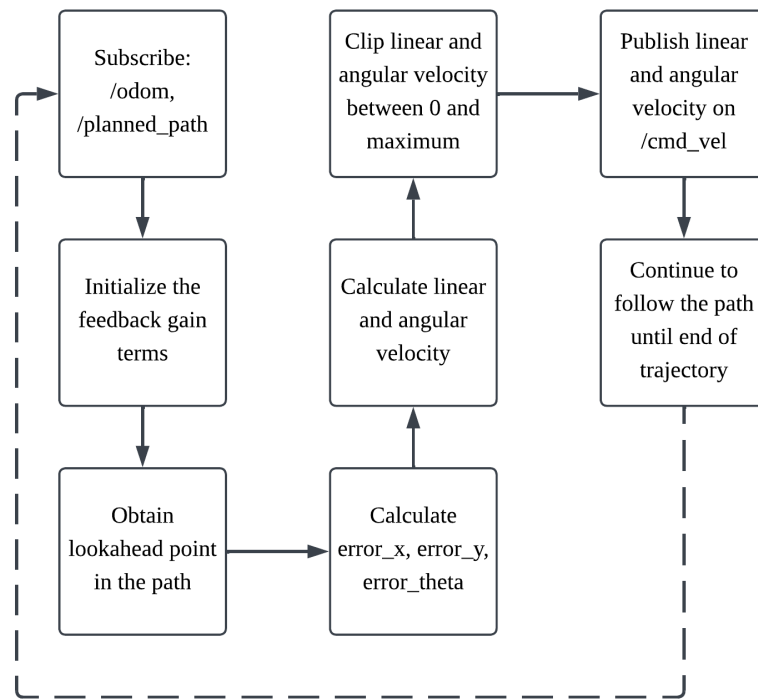


Figure: Gazebo house simulation environment with turtle bot 3

Task 1:



Obtain new path from /generated_path when used along with A Star Planner

Figure: Block diagram describing the process flow of robot controller

1. Non-Linear positional controller with look ahead was used as the controller of choice to provide the mobile robot linear and angular velocities to perform point to point traversal and trajectory following.
2. The pose errors and orientation error were calculated:

$$e_x = target_x - robot_x$$

$$e_y = target_y - robot_y$$

$$e_theta = wrap(arctan2(e_y, e_x) - robot_theta)$$
3. Angular velocity and linear velocity were calculated

$$Distance = Euclidean_distance(e_x, e_y)$$

$$Angular_velocity = k_y \times e_theta$$

$$Linear_velocity = k_x \times distance \text{ (if } e_theta < 0.4 \text{ else } 0)$$

4. The angular and linear velocities were clipped to prevent overshoot of velocities
 5. The robot would produce angular and linear velocities for the look ahead points obtained from querying the trajectory
 6. In the look ahead logic, the points which have a distance greater than the look ahead distance in the trajectory were taken as the target points
 7. The robot would execute the controller and look ahead logic until the goal is reached and then it would stop publishing the angular and linear velocities.
- 8. Pose-to-Pose Navigation:**
- a. For Pose-to-Pose navigation task a path generation script generated a fixed goal point at (5,3) and an orientation of 30 degrees. It is set by the user.
 - b. Intermediary poses are created between the final pose and the start pose for the robot to navigate to the goal pose
 - c. Initially tried to perform pose-to-pose navigation using just the start and end pose, the robot kept going off track. The code for intermediary poses was given by Claude AI
 - d. The robot controller accepts the goal position and orientation and produces
- 9. Circular path navigation:**
- a. For circular navigation the radius of the circle was chosen to 2.0 and number of points on the circumference was set to 72 (every point is 5 degrees away)
 - b. The points on the circumference of the circle were obtained and converted into ros messages of PoseStamped() and given to the controller.

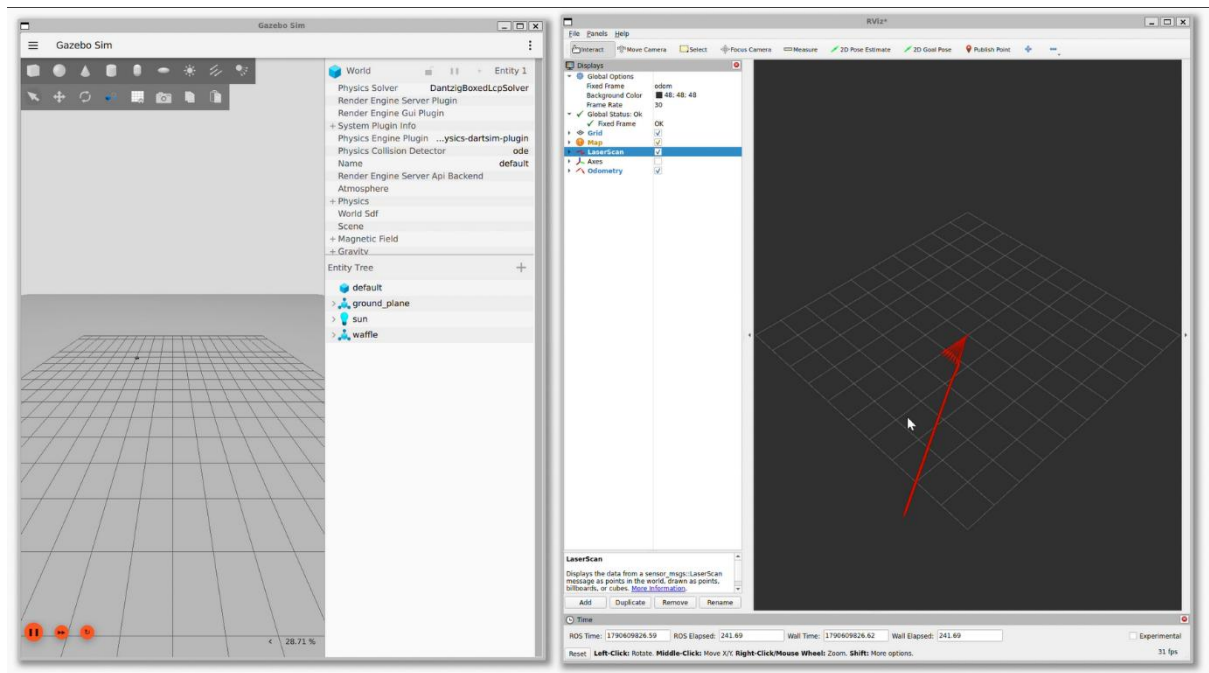


Figure: Gazebo and Rviz visualization output for Pose-to-Pose navigation

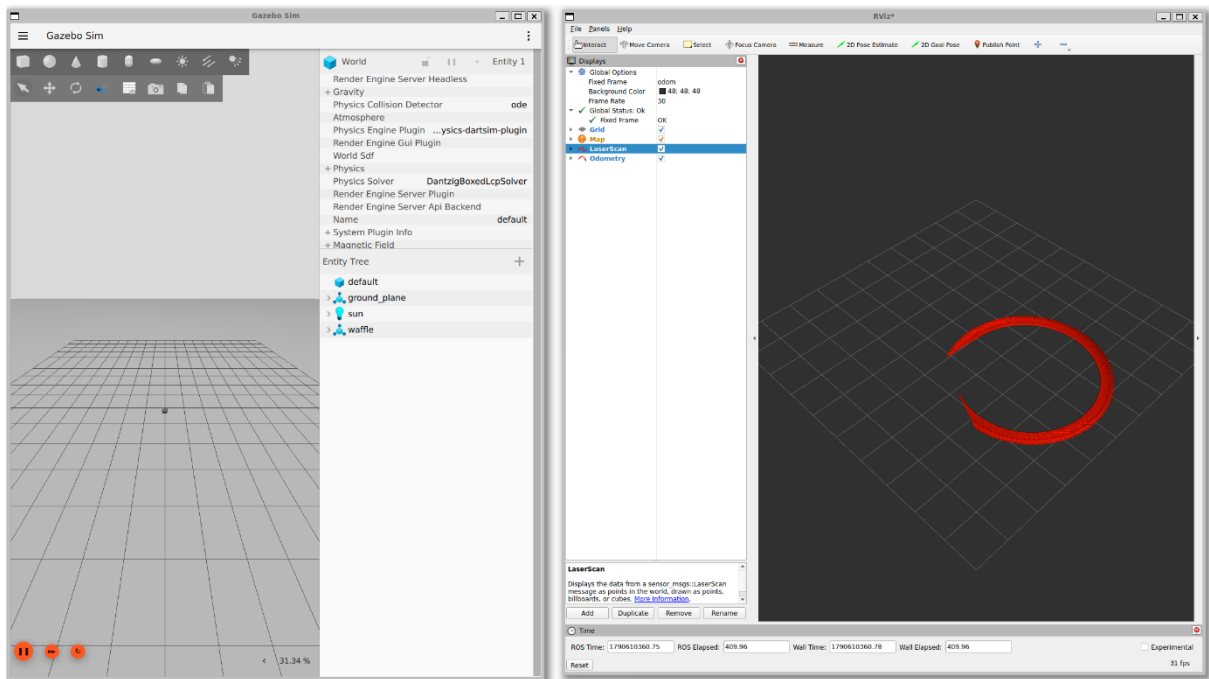


Figure: Gazebo and Rviz visualization output for Circular navigation

10. A small change was done to the robot controller used to receive frontier points. As frontier points did not have any inherent direction the yaw angle was set at 0.0 in the quaternion orientation from the A star planner.

11. The control logic is calculated as follows:

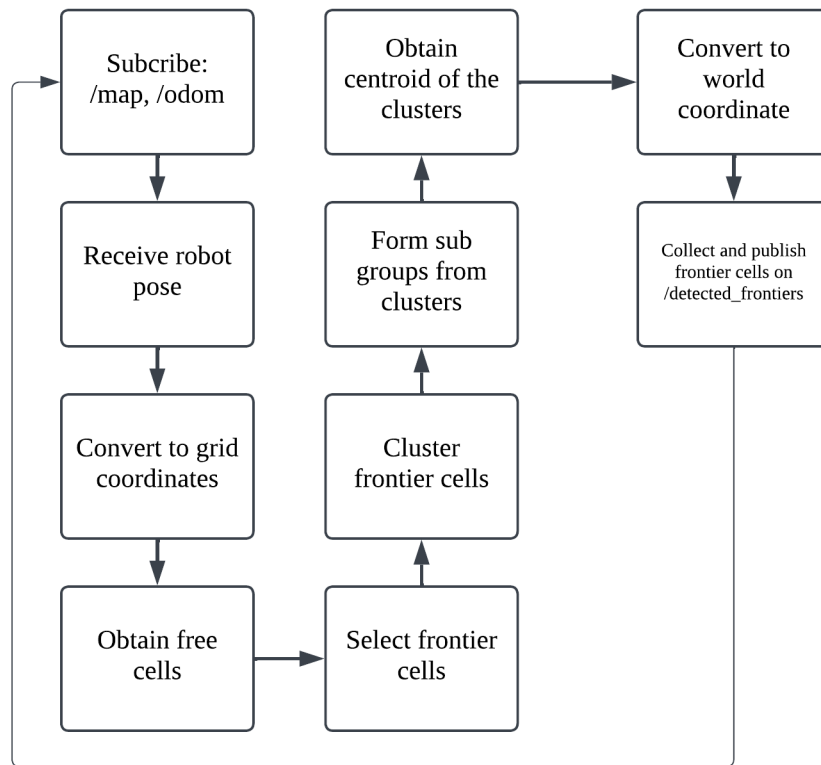
$$\begin{aligned}e_x &= target_x - robot_x \\e_y &= target_y - robot_y \\distance &= euclidean_distance(e_x, e_y) \\angle_to_target &= arctan2(e_y, e_x) \\e_theta &= angle_to_target - robot_theta \\e_theta &= arctan2(\sin(e_theta), \cos(e_theta)) \\angular_velocity &= k_y \times e_theta \\linear\ velocity &= k_x \times distance \text{ (if } e_theta < 0.4 \text{ else } 0)\end{aligned}$$

12. The initial experiment with cartesian errors (difference between the robot pose and the target pose) landed the robot in a stalled state where it would rotate about an axis and would never move forward. A polar form calculation where the distance between the robot pose and the target pose calculated and the error pertaining to the heading was warped to $(-\pi, \pi)$ provided with a smoother traversal to the goal position. This change was suggested by Claude AI

Task 2:

1. For the task of exploration nav2's online slam package was used. It was launched by ***ros2 launch slam_toolbox online_async_launch.py***.
2. The slam tool box utilized the 2D scans from the turtle bots LiDAR scanner to produce a 2D map of the environment by utilizing a pose-graph constructed by the robot pose and the scans.
3. It uses asynchronous method to process the scans where scans are processed independently from the sensors acquisition cycles
4. Although the occupancy grid map was not the complete map of the environment. It would create an occupancy grid as far as the maximum range of the LiDAR scanners.
5. It produces ***/map*** to ***/odom*** coupling while creating the map
6. The map would have to be built subsequently when the robot traverses the environment

Frontier Points:



New frontier points are updated as they are available

Figure: Block diagram describing the process flow of frontier point generation

1. The subscribers in this function were:
 - a. /map: The occupancy grid map produced by the slam algorithm
 - b. /odom: The odometry of the robot
2. The publishers in this function were:
 - a. /detected_frontiers: The detected frontier points
3. Frontier points correspond to the cells on the grid map which are present at the boundaries of free cells and unknown cells.
4. The frontier points are obtained by taking into consideration the points at the boundaries of free cells and unknown cells. This in practice produces a larger number of frontier points in orders of hundreds. Smaller the resolution of the occupancy grids the larger will be the boundary points too.
5. The robot does not require a larger number of frontier points in the magnitude of several hundreds. Hence once the frontier points are obtained, they are clustered to reduce the number of frontier points. This significantly reduces their number.
6. In this particular case the, the numbers reduce from frontier points of about 487 to numbers of 2, 3, 4, 7 or 8 depending on the position of the robot.

7. The exceedingly small number of frontier points seemed simple to work with initially although it proved cause issues with the robot exploration,
8. At times when the number of boundary points were 2 or 3 the robot would move to these 2 or 3 points and come to a halt as the map would not be updated with the LiDAR scans to obtain new frontier points as they were quite close to each other
9. In this case the clusters formed were sub grouped with a chosen maximum sub group size (5 in this case, can be other sizes too, just make sure the number is not too small as it would lead to large number of frontier points) and their centroids were calculated and used as frontier points. This produced frontier points in the range closer to about 50. It would change as the robot explored.
10. Certain frontier points created would be surrounded by unknown cells, this led to a situation that a search algorithm such as A star was not reachable to the points making the point redundant.
11. In this case, the frontier points were checked to see if they are suitable as a candidate by checking their 4 neighbors and only then they were provided to the algorithm to be used as suitable frontier points.
12. The frontier points were published as a Pose Array on the topic ***/detected_frontiers*** which was to be utilized by the search algorithm to create a path.

A star search algorithm:

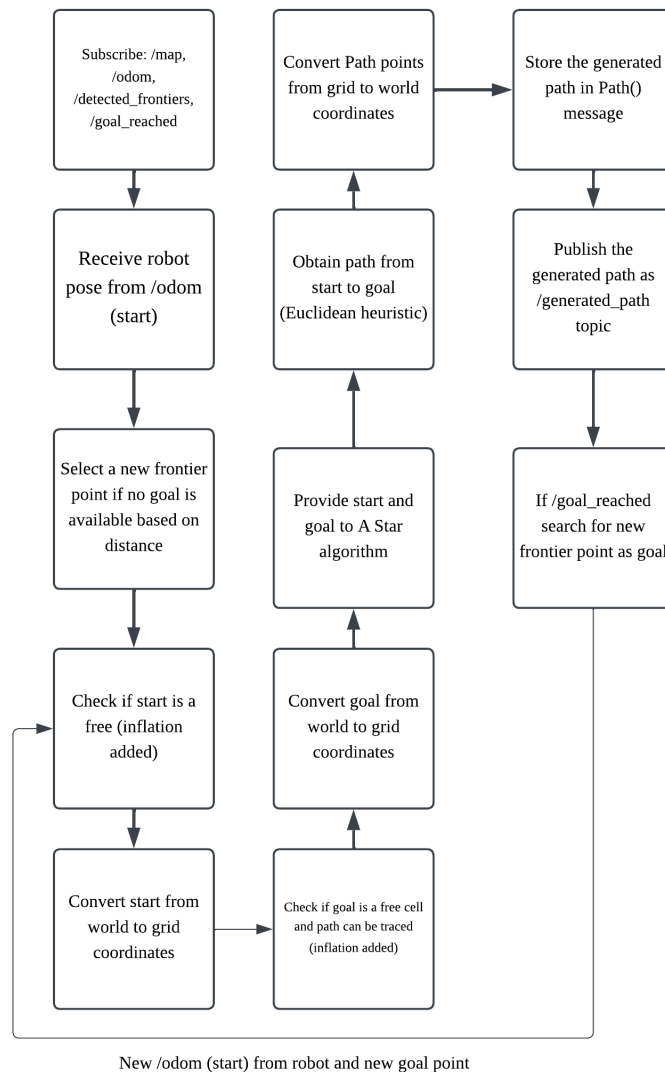


Figure: Block diagram describing the process flow of A Star algorithm and path generation. Frontier point selection is also done in this algorithm

1. The subscribers in this function were:
 - a. /map: The occupancy grid map produced by the slam algorithm
 - b. /odom: The odometry of the robot
 - c. /detected_frontiers: The detected frontier points
 - d. /goal_reached: A goal reached trigger from the robot controller
2. The publishers in this function were:
 - a. /planned_path: The planned A star path from current robot pose to the selected frontier point to function as the goal.
3. The A star algorithm and planning formed one of the core functionalities of the exploration algorithm. At its core sits the A star search algorithm created for 8

neighbors. The algorithm takes in a start and a goal point, where start corresponds to the current pose of the robot and goal corresponds to the frontier point selected.

4. The frontier point in this scenario is selected to the one having the smallest Euclidean distance from the current pose of the robot.
5. The code section subscribes to the **/odom**, **/map**, **/detected_frontiers** and a **/goal_reached** signal from the robot controller. It publishes the **/path** for the robot controller to follow.
6. The frontier point closest to the robot is select to be the goal. If the points end up to be in not a free state due to obstacle inflation, its neighboring points are used. A similar method is followed for the start point.
7. Once the frontier point is selected it is added to a list containing reached frontier points as it should not be utilized again
8. A star algorithm accepts the start and goal position and produced a grid path. Components of the grid path and converted into world coordinates and then published as a Ros path
9. The robot controller utilizes this path and using a look ahead method publishes linear and angular velocities to the robot.
10. Once the robot has reached its goal, it sends a signal on a published topic stating that it reached its goal and asks the planner to provide a new path
11. The planner searches the frontier list and provides the A Star algorithm with a new frontier point to produce a new path.
12. By following the same procedure, the robot is able to explore the environment it is present
13. The planner is called every 2 seconds to create the A Star path. While doing this the goal does not change as it can be changed only by the **/goal_reached** signal provided by the robot controller. The start position of the robot though changes.

Overall system working:

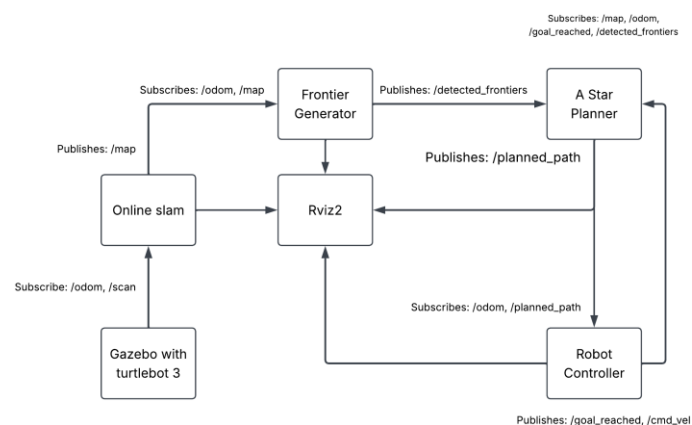


Figure: General block diagram of communicating parts of the overall system with descriptions of publishers and subscribers

1. The turtle bot is simulated in a Gazebo environment.
2. The asynchronous online SLAM algorithm creates a Occupancy grid map available.
3. The Frontier generation code uses this map to produce frontier points
4. The A star planner utilizes the frontier points to generate a feasible path from the current robot pose to the frontier point. It runs the A star plan every 2 seconds and publishes the path.
5. The robot controller utilizes the generated path by A star to reach the frontier points. Once the frontier point is reached it sends a /goal_reached signal to obtain a new trajectory
6. The visualization of the occupancy grid, frontier points, path is done in Rviz2

Map Generated:

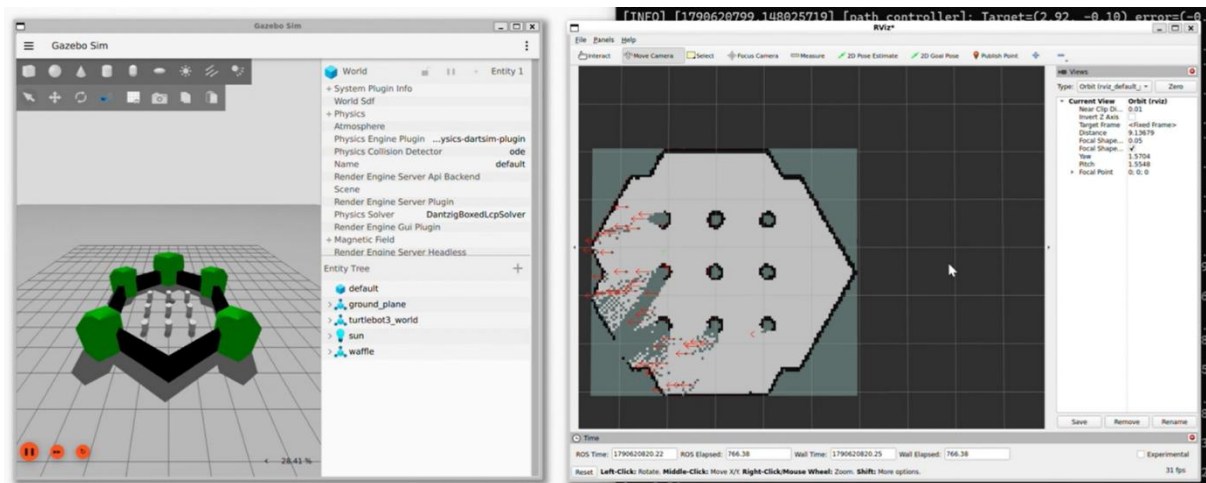


Figure: Image of the Gazebo turtle bot world on the left side and the map generated by frontier exploration on the right side. The arrows mark the frontier points. The green path is the trajectory

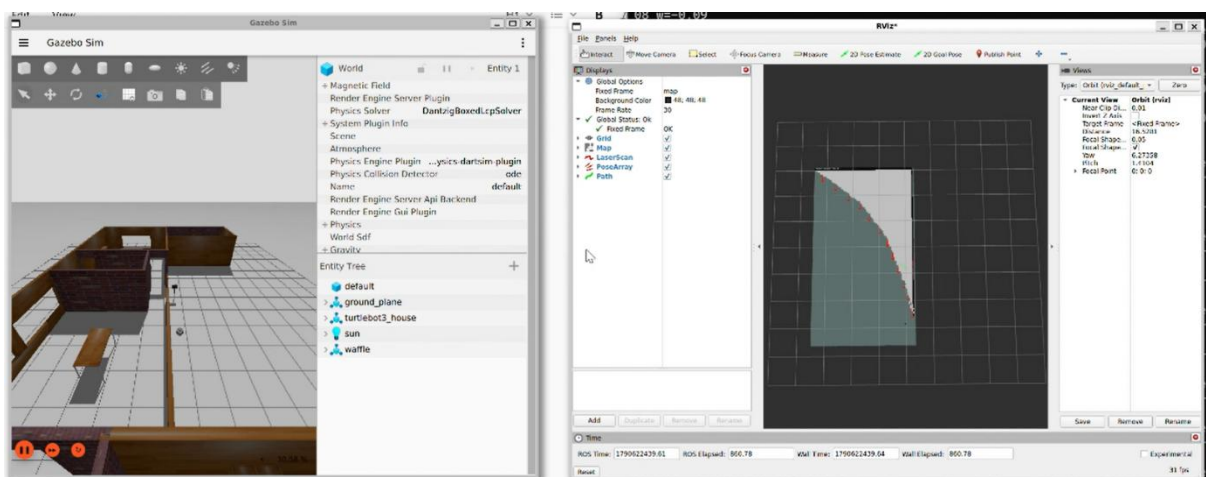


Figure: Image of the Gazebo House on the left side and the map generated by frontier exploration on the right side. The arrows mark the frontier points. The green path is the trajectory

1. The robot explored quite well in the Gazebo turtle bot environment as indicated by the map. Although at the end my system started to stall as the simulation was running on the CPU hence the results would start to become erratic as can be seen at the end of Gazebo_turtle_bot_world environmnet. The original video was about 10 minutes long. It is fast forwarded in the submitted video due to storage constraint.
2. The robot exploration in Gazebo house environment was not very great but given sufficient time I believe it would have explored the environment. The original video was about 13 minutes long.
3. An issue that was observed is that the robot at time does not generate a path to the visible frontier points in the environment. It would generate intermediary path. I believe this is probably caused due to the robot reusing the old frontier points. A better flag check could prevent this.

Discussions from the experiments:

1. The robot controller is a non-linear positional controller and does not drive in smooth curves as a pure pursuit controller would.
2. Initially tried to use Nav Planner using a yaml file to produce A star trajectories to frontier points but the planner would not function even with the correct yaml configuration. Hence, a switch was to done to a A Star-based planner.
3. The core A Star based planner was given by ChatGPT but additions were performed by me to selection of frontier points the frontier points planner section to accept /goal_reached when the controller would reach the final goal.
4. A probable bug observed in the frontier navigation is that at times the robot does not correctly form a path to the frontiers displayed in the Rviz environment. I believe this is probably caused due to the robot reusing the old frontier points. A better flag check could prevent this.
5. Some of the videos were not fully completed as my system started to stall as the simulation ran on the CPU in WSL.

Task 4:

1. A drawback in this project would be the lack of consideration on the structure and geometry of the robot as it is necessary while traversing an environment.
2. An addition of Cost map would prove to be a improvement to the planning algorithm.
3. A planner such as a pure pursuit based one would enable the robot to move smoothly about the environment.
4. The focus here is only laid on the exploration aspect of the robot hence only a 2D map is constructed. In real world scenarios a 3D map would be very suitable especially when exploring unknown environments where it would be possibly dangerous for humans to explore.

5. Dynamic obstacle avoidance was not incorporated into the system. It would considerably improve the navigation capabilities in an environment.
6. Only a single robot is used in exploration and thus consumes a larger amount of time to completely explore the environment. Use of multiple robots would be very beneficial
7. One the topic of multi robot navigation use of ariel vehicles could drastically improve the capabilities to work with high altitude structures such as bridges.
8. A UGV could map the ground terrain and a UAV could map structures at high altitudes

Paper Idea:

1. I found the robot to be very slow during the task of exploration hence a paper suggestion would be *“A Novel Controller Producing Safe and Higher Velocity Commands for motion planning in a Multi-Robot Exploration Setting for UGV’s in a Highly Dynamic Environment”*
2. Use of Control Barrier Functions would enable safety in the robot velocity output. Control Barrier functions also aid in dynamic obstacle avoidance by incorporating a safety barrier around moving objects by considering its orientation, velocity and position.
3. Control Barrier functions along with Model Predictive Controllers could provide a safe region of navigation for robots with higher velocities

Use of AI:

1. ChatGPT and Claude AI were used during the work of the project
2. The initial robot controller was designed using positional error which caused the robot to jitter and Claude provided with a solution of utilizing polar coordinates-based approach and provided the code for it.
3. The initial path generation code disregarded intermediate points when robot navigated pose-to-pose. Claude provided the code segments for intermediary point generation
4. The frontier exploration code was provided ChatGPT. The addition of multiple subclusters to produce more clusters was done by me
5. The core A Star algorithm was provided my ChatGPT. I worked on the creation of frontier selection and frontier point switching in the planning section.
6. ROS Wiki was used to obtain more information regarding general operation in ROS.